

RAG

介绍一下RAG，Agentic RAG和GraphRAG，以及他们的区别

这三者是 RAG 家族的三条演进路线，**不是升级阶梯，而是针对不同类型问题的三种解法**——单跳事实用 Classic RAG，多跳关系用 GraphRAG，复杂多步多源用 Agentic RAG，生产里经常混合编排。

下面分别拆开讲。

Classic RAG（检索增强生成）

核心动词：Retrieves（检索）

最经典的"用户问 → 向量检索 → 拼上下文 → LLM 生成"单程流水线。

- **索引侧**：文档切块 → embedding → 存入向量库
- **查询侧**：问题转向量 → 相似度召回 top-k 块 → 喂给 LLM 生成
- **强在哪**：实现快、成本低、对"《员工手册》里年假多少天"这种单跳事实查询够用
- **弱在哪**：靠语义相似度，**看不见实体间的关系**。一旦答案需要把散在多处信息串起来，就容易漏召回或拼错

GraphRAG（图谱增强）

核心动词：Connects（连接）

在 Classic RAG 之外**叠了一层知识图谱**，专治"向量检索找不到关系链"的毛病。

- **索引侧**：LLM 从文档里抽实体（人/系统/概念/事件）和关系，建成图
- **检索侧**：不只是向量相似，**还做图遍历**——沿着实体-关系-实体一路跳
- **经典例子**（很能说明问题）：

向量库三条记录：①"结账服务用支付 API" ②"支付 API 跑在 cluster-3" ③"cluster-3 周五维护"

问："结账服务会受周五维护影响吗？"

向量检索能召回①③（都含关键词），但②**在嵌入空间里离查询太远**，既没"结账"也没"维护"，直接漏掉 → 推不出完整链路。

GraphRAG 靠图遍历一次性把"结账→支付API→cluster-3→维护"全串起来

- **适合**：合同审查、政策解读、影响分析、组织知识网络这类**关系稠密**的场景

- 代价：建图抽实体依赖 LLM 质量，增量更新图谱维护重，单跳事实场景杀鸡用牛刀

Agentic RAG（智能体化）

核心动词：Reasons（推理）

不是换检索结构，而是把检索流水线交给 Agent 管——从"被动检索"变成"主动决策"。

- Classic RAG 是线性静态的：检索一次 → 生成，检索逻辑写死
- Agentic RAG 是迭代式的：Agent 接到问题后——
 - a. 先规划：这题要查哪些源、要不要多跳、要不要调工具
 - b. 动态选工具：语义搜索 / SQL 查结构化库 / Web 搜索 / API 调用，随便组合
 - c. 看中间结果：不够就再查、换策略、验证来源、反思重写
 - d. 出答案时带完整溯源和审计日志
- 适合：法律分析、排障、财报研判这类开放、多步骤、要跨多源的任务
- 代价：延迟高、token 成本高、Agent 编排不好容易陷入思考死循环，评估也比固定管线难

三者对照

维度	Classic RAG	GraphRAG	Agentic RAG
核心机制	向量相似度	向量 + 图谱遍历	Agent 动态规划 + 多工具
解决痛点	事实检索	多跳关系推理	复杂多步多源任务
推理能力	弱	中（有结构路径支撑）	强（有反思/再规划）
成本/延迟	最低/最快	中等	最高/最慢
维护负担	低	图谱构建+更新重	Agent 编排+评估难

选型的一句话建议

-  不是越新越好，而是越对症越好：
- 80% 的 FAQ / 内部文档问答 → Classic RAG 够了，Hybrid（向量+关键词+元数据）更稳
 - 知识本身是一张网（合同、规章、运维拓扑）→ GraphRAG
 - 问题类型没法预设、要查数据库还要上网还要算一步 → Agentic RAG
 - 真实项目里三者常叠加：Hybrid 检索做底 + Graph 补关系 + Agent 管复杂查询

RAG面试中常见问题

一、基础概念层（几乎必问）

为什么要用 RAG，不直接微调 / 不用长上下文？

- RAG：知识可实时更新、可溯源、训练成本低，适合外部知识频繁变动
- 微调：学的是「能力/风格」，不是「事实记忆」，更新要重训
- 长上下文：128K 看起来能塞，但噪声丢信息、成本炸、定位不准，RAG 做预过滤仍是主流

RAG 的标准流程是什么？

文档加载 → 清洗 → 切分 → Embedding → 入库 → Query 改写 → 检索 → Rerank
→ 拼接 Prompt → LLM 生成

Classic RAG 有什么问题？

- 切分粗糙丢上下文
- 检索靠相似度 ≠ 相关性，top-k 噪声大
- 没有 Query 理解，问"A 对 B 的影响"检索回一堆 A 和 B 的单边文档
- 没有溯源、没有拒答，幻觉照样出

二、检索侧（考察最深的一段）

切分（Chunking）策略有哪些？怎么定大小？

固定长度切分（Fixed-size Chunking）

最朴素的做法：按 token 数 / 字符数一刀切，比如每 256 token 一块，块间留 10%~20% overlap。

-  实现零成本、batch 均匀，训练/索引都友好
-  完全无视语义边界，一句话拦腰斩、一个表格劈两半是常态
- 适用：原型验证、对精度要求不高的场景

💡 Overlap 一般设 chunk 的 10-20%，目的是让"被切断的那句话"在相邻块里还能凑出完整上下文。

递归字符切分（RecursiveCharacterSplitter）

LlamaIndex / LangChain 默认方案，先按 `\n\n` 切，不行再按 `\n`，再不行按 `。`，再不行按空格……层层降级到固定长度。

-  比纯固定长度聪明很多，优先保段落/句子完整
-  对 Markdown、普通文档鲁棒性不错
-  仍然不懂"代码函数""表格"这种特殊结构
- 适用：通用文档（PDF、Word、网页）80% 场景的起步选择

结构感知切分（Structure-aware）

按文档原生结构切，而不是按字符数：

- **Markdown**：按 `#` 标题层级切，标题作为 metadata 继承下去（"二级标题=薪酬"的那块，检索时能加权）
- **代码**：按 AST 切——函数/类/方法为单位，而不是按行数。树莓派官网那套文档尤其明显，切到函数级检索精度远高于按 500 token 硬切
- **HTML**：按 DOM 节点 / `<div>` / `<section>` 切
- **PDF**：先用版面分析（Unstructured、LlamaParse）分出"段落/表格/页眉页脚"，再切

面试加分点：代码 RAG 如果按字符切，召回的函数永远缺一半，`def` 在这块、`return` 在下块——这是高频翻车现场。

语义切分（Semantic Chunking）

不是看字符，而是看 **embedding 相似度** 决定在哪断：

- 思路：把文档按句子滑窗，相邻句子 embedding 算余弦，相似度**跌下来**的地方就是话题切换点，在那切
- 代表：LlamaIndex 的 `SemanticSplitterNodeParser`、Jina 的 `SemanticChunker`
-  切出来的块内部话题高度一致，检索时"整块相关"的概率高
-  要用 embedding 模型过一遍全文，预处理成本高；阈值要调

适合：长文、报告、论文这种"一段一个话题"的文体。

命题切分（Proposition Chunking）

把一段文本**拆成若干条原子命题**存，而不是存原文块。

例子：

原文："小明是张三的儿子，今年 8 岁，在北京上学。"

拆成：

- 小明是张三的儿子

- 小明今年 8 岁
- 小明在北京上学
- ☒ 检索精度极高，LLM 生成时噪声最小
- ☒ 特别适合事实密集型（百科、合规条款）
- ☒ 完全依赖 LLM 拆，成本、延迟都上去了，拆错一条就埋雷
- 代表：Dense X Retrieval 那篇论文提的思路，LlamaIndex 有 `PropositionSplitter`

按"父-子"切分（Parent-Child / Sentence Window）

检索用小的，生成用大的——两套 chunk 配套：

- **Child**：句子级 / 小段，用来做 embedding 检索（精细）
- **Parent**：Child 所属的整段 / 整节，检索命中后把 Parent 拼给 LLM（上下文完整）

变体还有 **Sentence Window**：检索命中等第 3 句，生成时把句中前后各 N 句窗口拼回去。

- ☒ 兼顾"检索准"和"生成上下文足"，工业界很常用
- ☒ 要存两套索引或加 metadata 回溯，复杂度高点

选型速查

场景	推荐
快速原型 / 通用文档	RecursiveCharacter（递归字符）
Markdown / 技术文档	结构感知（标题层级）
代码库	AST 级结构切分
长报告 / 论文	Semantic Chunking
合规 / 百科类高精度	Proposition + Parent-Child
对话式 QA	Sentence Window

面试如果被追问"chunk size 设多少"，标准答法是：没有银弹，256-512 token 是起步经验值，但最后要靠 Recall@k 在验证集上跑出来——QA 偏短、摘要偏长、代码按函数。能补一句"我们同时跑了 Rerank，所以召回侧可以稍宽、精排再筛"会更像实操过的。

不同切分怎么配 Embedding 模型

核心原则：切得越细、语义越"碎"，对 embedding 模型的语义理解力要求越高；反之切块大、冗余多，轻量模型也能扛。

固定长度 / 递归字符切分 → 通用稠密模型即可

块大（512~1024 token）、内部上下文自治性强，相似度算糙一点也能召回。

- 推荐：**BAAI/bge-large-zh-v1.5**、**text-embedding-3-small**、**cohere-embed-multilingual**
- 理由：块里信息冗余多，"年假 10 天"哪怕切块里只露半句，靠周边词也能被 vector 捞回来
- 成本友好，量产首选

语义切分（Semantic Chunking）→ 同款通用模型，但要和切分时用的 embedding 同源

语义切分本身就是靠 embedding 相似度跌宕来断句的，检索时如果换一个 embedding 空间，断层位置可能和检索空间对不齐。

- 建议：切和检用**同一个 model**，`bge-large` 切 + `bge-large` 检，别 `text-embedding-3` 切 + `bge` 检
- 这是个面试小坑，能提一句显得细

命题切分（Proposition）→ 必须用高精度 embedding，最好是 rerank 也跟上

命题已经是原子级了（"小明 8 岁"），信息密度极高、无冗余——这时候 embedding 要是语义理解弱，稍微换个说法就召回不回。

- 推荐：**bge-large-zh / bge-m3**、**text-embedding-3-large**、**jina-embeddings-v2**
- m3 和 jina-v2 支持 8192 上下文，虽然命题短用不上，但他们对细粒度语义区分更好
- 命题切 + 轻量 embedding（如 all-MiniLM）= 自废武功，常见翻车组合

父-子切分（Parent-Child）→ Child 用轻量，Parent 可不上 embedding

- **Child（检索用）**：句子级，信息碎 → 用**精度高一档**的模型（同上命题切那档），因为 child 要靠自身语义被捞，没有 parent 兜底
- **Parent（生成用）**：不进向量库，靠 child_id 回溯拿文本即可，省一份 embedding 成本
- 或者反过来玩：**Child 用 BM25 检（省 embedding）** + **Parent 用向量做二次粗排**——多路混合

代码 AST 切分 → 专用或通用大模型都可，但 dim 要够大

代码语义靠标识符+结构，通用中文模型（bge）其实也能打，但：

- 更好选择：**CodeSearchNet** 系、**unixcoder**、**jina-embeddings-v2-code**
- 关键不是模型多"专"，是 **max_seq 要够吞下整个函数**——代码切出来单块可能 1k+ token，选 512 截断的模型直接废

表格 / 多模态 → CLIP 类或表格专用

- 表格：转 Markdown 后当文本用 bge-m3 也行；讲究点用 **TableGPT**、**TaBERT** 思路
- 图文：**clip-vit**、**alt-clip**，或者新一点的 **jina-clip**

选型速配表

切分方式	块大小	embedding 选型倾向	原因
固定/递归	512+	通用轻量 (bge-base、text-embed-3-small)	冗余多，糙点没事
语义切	200~500	同一切分模型（同源）	空间对齐
命题切	30~80	高精度 (bge-large、text-embed-3-large)	原子级无冗余
父-子	child 短	child 用高精度，parent 不检	child 自治性差
代码 AST	可变	长上下文+代码向 (unixcoder、jina-code)	吞得下整个函数
表格	—	m3 通用 / Table 专用	结构语义

 面试加分提法：**embedding 模型不是越新越大越好，是和 chunk 的"自治性"成反比——chunk 越碎越要砸好模型。**

Chunk 元数据怎么设计

metadata 才是 RAG 从 demo 走向生产的关键。同样的 chunk 文本，metadata 设计得好，检索侧能玩出花；设计得差，rerank 再强也补救有限。

必选基础字段（每条 chunk 都要有）

代码块

```
1  {
2      "chunk_id": "doc_123_chunk_07",
3      "doc_id": "doc_123",
4      "doc_title": "2024员工手册.pdf",
5      "chunk_index": 7,
6      "total_chunks": 42,
7      "text": "...",
8      "created_at": "2024-01-15"
9  }
```

- `chunk_index` + `total_chunks`：做 sentence-window / parent-child 回溯时必备
- `doc_id`：溯源、去重、权限过滤

结构感知字段（强烈建议）

代码块

```
1  {
```

```

2     "section_h1": "薪酬福利",
3     "section_h2": "年假规定",
4     "page_num": 12,
5     "block_type":
6     "paragraph"    // paragraph / table / code / image_caption
7 }

```

- `section_h*`：检索后可以**层级 boosting**——“年假”问题里，命中 `h2=年假规定` 的块权重 $\times 1.3$
- `block_type`：表格和正文混在一起时，检索后可以分开处理（表格走 markdown 渲染，代码走 AST 高亮）
- `page_num`：给用户展示溯源时巨重要

父子 / 窗口回扩要用到的字段

代码块

```

1  {
2     "parent_id": "doc_123_chunk_05", // 自己是 child 时指向 parent
3     "window_before": 2, // 前 2 句
4     "window_after": 2  // 后 2 句
5  }

```

- Sentence Window 玩法：检索命 `chunk_07`，生成时拿 `chunk_05~09` 拼一起
- Parent-Child 玩法：检索命 child，生成拿 parent（`parent_id` 回溯）

检索侧能当 filter / boost 的业务字段

代码块

```

1  {
2     "dept": "HR",
3     "doc_type": "policy",          // policy / faq / tech_doc / contract
4     "effective_date": "2024-01-01",
5     "expires_at": "2025-12-31",
6     "tags": ["年假", "加班", "外派"],
7     "access_level": "all"         // all / L3 / L4 多租户权限
8  }

```

用法举例：

- **Metadata Filtering**：`doc_type=policy AND dept=HR`，向量检索前先拦一刀，比纯语义稳

- **时间衰减**: `effective_date` 旧的降权, 过期文档直接 filter 掉 (比 rerank 拦早一步)
- **Tag boost**: 用户问"年假", tag 含"年假"的块 $\times 1.2$
- **权限**: 多租户场景 `access_level` 必须带, 检索完再拦一层也不如索引层就 filter 掉

多模态 / 高级字段

代码块

```
1  {
2      "has_table": true,
3      "table_caption": "表3-2 各职级年假天数",
4      "image_ocr": "ALT 文本或 OCR 结果",
5      "language": "zh",
6      "embedding_model": "bge-large-v1.5" // 防以后模型迁移混库
7  }
```

元数据设计的三个原则 (面试能说的"方法论")

1. **检索前能 filter 的, 别留给 rerank**——`doc_type`、`dept`、`expires_at` 这些在向量库层面就用 metadata filter 干掉, 省 token 省延迟
2. **检索后能 boost 的, 别只靠向量分数**——`section_h2` 匹配、tag 匹配、`chunk_index` 连续 (同一章节的多块命中说明这章是重点) 都可以当信号加权
3. **溯源链要完整**——`doc_id + chunk_index + page_num + section` 四个拼起来, 用户问"你这话哪来的", 能精确到"员工手册 p12 年假规定第二节", 信任感一下就上来

什么是BM25检索

BM25 是搜索引擎里那套**基于关键词的稀疏检索算法**, Elasticsearch / OpenSearch / Lucene 底层默认就是这个。在 RAG 语境里, 它和向量检索是一对互补——**向量抓语义, BM25 抓原词匹配**。

它是谁、从哪来

BM25 = **Best Matching 25**, 属于"概率信息检索"流派, 是 TF-IDF 的改进版。核心思想没变——**一个词在某文档里出现越多越重要, 但在全库出现越多越不重要**, 但 BM25 在 TF-IDF 两个弱点上打了补丁:

1. **TF 不是线性涨** —— 词出现 1 次和 2 次差别大, 出现 50 次和 51 次不该再差太多 (饱和函数)
2. **文档长度要归一** —— 长文档天生容易命中更多词, 不能吃这个便宜

直觉版公式

不用背，面试能说清三个部件就行：

$$\text{Score}(q,d)=t\in q \sum \text{IDF}(t)\cdot f_t+k_1\cdot (1-b+b\cdot L_{\text{avg}}L_d)f_t\cdot (k_1+1)$$

拆开看每个符号在干嘛：

- **IDF(t)**：逆文档频率， $\ln((N - n_t + 0.5)/(n_t + 0.5) + 1)$ ，全库都有的停用词 ("的""是") $\text{IDF}\approx 0$
- **f_t**：词 t 在文档 d 里出现次数 (TF)
- **k₁**：控制 TF 饱和度，一般 1.2~2.0。k₁ 越大，词频越能拉开分差
- **b**：控制长度归一化强度，0~1，一般 0.75。b=0 不归一，b=1 完全按长度比归一
- **L_d / L_{avg}**：当前文档长度 / 全库平均长度

💡 分子 (k_1+1) 是为了让 $\text{TF}\rightarrow\infty$ 时整项趋近于 $(k_1+1) * \text{IDF}$ ，保证饱和有个天花板。

在 RAG 里为什么还要用它

现在 LLM 时代为啥还要聊一个"非 AI"的老算法？因为**向量检索有它搞不定的场景**：

场景	向量检索	
"报错码 ERR_429 什么意思"	❌ "ERR_429"是低频专有词，embedding 训练时没见过，向量空间里漂得乱七八糟	✅
"iPhone 15 Pro Max 价格"	❌ 型号名、产品代号，语义相似度不一定高	✅
"离职和辞职有什么区别"	✅ 语义能兜底"离职≈辞职"	❌
同义词/ paraphrasing	✅ 强项	❌

所以工业界 RAG 标配是 **Hybrid：向量召回 + BM25 召回 → RRF (Reciprocal Rank Fusion) 或加权合并 → Rerank → LLM**。

和向量检索的本质区别

- **BM25 = 稀疏、字面、确定性**：词袋模型，不管语序，不管语义，但**可解释、可调、对专名 ID 绝对可靠**
- **Vector = 稠密、语义、模糊性**：embedding 空间里"离职"和"辞职"靠得近，但"ERR_429"如果训练语料没见过就废

面试还能补一句：**BM25 不需要训，向量需要训；BM25 零样本直接上岗，向量要选模型 + 调维度 + 管版本**。

参数怎么设 (ELK/Lucene 默认)

- $k_1 = 1.2$ (TF 饱和拐点)，技术文档偏术语可以提到 1.5~2.0
 - $b = 0.75$ (长度归一)，短文本 (FAQ) 可以降到 0.3~0.5，长文 (合同) 保持 0.75
-

面试高频追问

Q: BM25 为什么叫 "25"?

A: 剑桥计算机实验室 Robertson 和 Sparck Jones 那票人，前后迭代了 BM1、BM2...BM25 是第 25 版，没啥玄学。

Q: RAG 里 Hybrid 的 RRF 是怎么融的?

A: $RRF_score = \sum 1/(k + rank_i)$ ，k 一般取 60，两个召回列表各自按排名倒数加权求和，不依赖各自分数的绝对值量级——这点比直接加权稳，因为 BM25 分和 cosine 分根本不在一个尺度。

Q: BM25 有什么缺陷?

A: 词袋模型无视语序 ("A 打 B" 和 "B 打 A" 对它一样)、无视语义、中文要先分词 (分词错就全错)——这也是为什么 RAG 不能只靠它。

检索为什么不全用向量，还要加 BM25 / 关键词?

- 向量擅长语义召回 ("离职流程" 能召 "辞职手续")，但**专有名词、ID、精确匹配**翻车
- BM25 补精确匹配，Hybrid (向量 + 关键词 + RRF 融合) 是工业界标配

Rerank 为什么必要? 和 Embedding 检索的区别?

- Embedding 是粗排，为了快，用内积/余弦，稠密向量对语义近似敏感但区分度有限
- Rerank (bge-reranker、Cohere、CrossEncoder) 是**精排**，Query 和 Doc 拼一起过一遍模型，算的是真·相关性，慢但准
- 工业化: Retriever 召 50 → Rerank 筛 5 → 进 LLM

Query 改写有哪些做法?

- 多轮对话里做 Query 指代消解 (把 "它" "这款" 还原)
- HyDE: 让 LLM 先生成假设性回答，再用假设回答去检索 (解决语义偏移)
- Step-back / Sub-query: 复杂问题拆多个子问，分别检

RRF 是什么?

RRF = **Reciprocal Rank Fusion (倒数排名融合)**，Cormack, Clarke, Büttcher 2009 年 SIGIR 那篇《Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods》提出来的，现在已经是 **Hybrid Search 的标配融合算法**——ES、OpenSearch、Azure AI Search、MongoDB Atlas、Weaviate 全都把它当默认 hybrid 融合方式。

上一轮我们把 RRF 嵌在管线第 3 站讲了，这次单独拆透它为什么存在、公式怎么读、k=60 哪来的这三层。

一、RRF 要解决什么痛点

Hybrid 召回做完，Dense 一路出 top-50、BM25 一路出 top-50，两路分数能直接加吗？

检索器	分数范围	问题
BM25	$0 \sim +\infty$ ，无上界	受 query 长度、词频影响，不同 query 分数不可比
向量余弦	$[-1, 1]$ 或 $[0, 1]$	和 BM25 量纲完全不在一个轴
Learned Ranker	$-\infty \sim +\infty$	和其他路更没法直接加

直接加权求和 = 量纲错位，谁分数大谁主导，不公平。

RRF 的核心洞察：一个检索器在自己列表里的排序是可信的，但分数的绝对值不可信。那就干脆扔掉分数，只认排名——排名是所有检索器共通的语言。

二、公式

$$RRF(d)=r\in R \sum k+rankr(d)1$$

- `rank_r(d)`：文档 d 在第 r 路召回里的排名（**1-indexed** 是论文原版；0-indexed 的实现会写成 `k + rank + 1`，本质一样，面试别被绕进去）
- `k`：平滑常数，默认 60
- 某路没召回到的文档，那路贡献 0，不加不减

直觉：**排名越靠前 → 倒数越小 → 分值越高**，但用 `k` 把"头部陡峭度"压住。

三、k=60 是哪来的（面试高频追问）

⚠ 这不是拍脑袋的经验值，是 Cormack 他们拿 TREC 评测集扫了 **k=1~100** 做出来的——k=60 在多数数据集上最优或接近最优，且 **k 在 40~80 之间性能差异 < 1%**，对取值不敏感。

k 控制的是什么？**头部排名的陡峭度：**

k	rank1	rank10	rank1/rank10 倍差	感觉
0	1	0.091	11×	头部独裁
10	0.091	0.048	1.9×	偏陡
60	0.0164	0.0143	1.15×	温和，共识优先
1000	0.001	0.00098	1.02×	全拉平

k=60 的效果：**rank1 只比 rank10 高约 15%，rank100 还有 rank1 的 38% 贡献**——RRF 奖励的是"多路都靠前"，不是"单路第 1 名碾压"。

💡 面试官那个梗："k=60 怎么来的，为什么不用 50 不用 100？"——标准答法是：Cormack 2009 TREC 扫出来的，40~80 都差不多，60 是论文报告的 sweet spot，对取值不敏感，生产不用动。

四、RRF 的行为直觉（为什么 Hybrid 用它）

举个两路例子，查询"苹果手机"：

文档	BM25 排	Dense 排	RRF(k=60)
文档A	1	2	$1/61 + 1/62 = 0.03252$
文档C	3	1	$1/63 + 1/61 = 0.03226$
文档B	2	4	$1/62 + 1/64 = 0.03176$

文档A 两路都前 2 → RRF 总分最高。这正是 Hybrid 想要的行为：

- 单路 #1、另一路拉胯 → 输给"两路都前 3"的
- 专名靠 BM25 捞、语义靠 Dense 捞，**两路都认的文档才配进 top**

五、变体：Weighted RRF

原版 RRF 每路权重都是 1（平等融合）。如果想偏重某一路（比如专名多的语料 BM25 权重高点）：

$$\text{WeightedRRF}(d) = \frac{1}{\sum_{r \in R} w_r \cdot k + \text{rank}_r(d)}$$

$w_r \in [0, 1]$ ，BM25 路 0.7 + Dense 路 0.3 是常见起点。

🚩 但工业界默认还是**非加权 RRF + k=60**——参数 free，多个系统（ES / Qdrant / Mongo）开箱即用，调 w_r 反而引入新超参要 ablation。

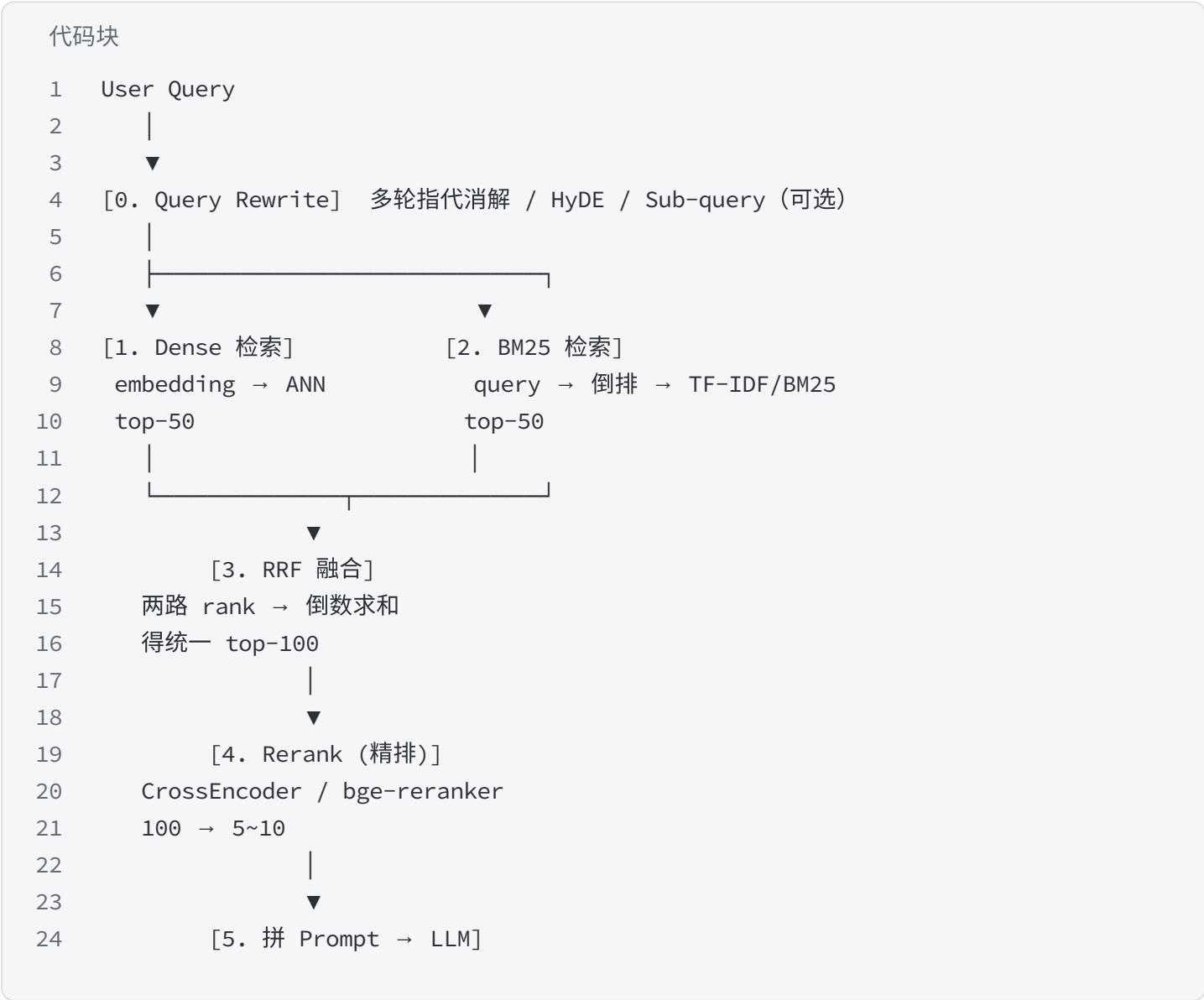
六、面试一句话闭环

RRF 是 Cormack 2009 提出的多路排名融合算法，抛弃跨系统不可比的原始分数、只认排名，用 $1/(k+rank)$ 倒数求和， $k=60$ 是 TREC 上扫 1~100 得到的 sweet spot。Hybrid Search 里"向量 + BM25 双路召回 → RRF 融合 → Rerank 精排"是生产标准三明治。

Hybrid Search + RRF 的完整召回管线是什么样子

Hybrid Search + RRF 这套管线，基本是生产级 RAG 召回层的"标准答案"——单路向量或单路 BM25 在面试里已经拿不到高分了，得能画出双路并行 + 融合 + Rerank 这条线。
下面按"请求从左到右跑一遍"的顺序拆。

完整召回管线（6 站）



每一站做什么

[0] Query Rewrite（可选但推荐）

复杂 query 直接进检索会翻车：

- 多轮："它支持退款吗？" → 指代消解成"XX 商品支持退款吗？"
- HyDE：LLM 先编一段假设答案，用假设答案去向量检索（缓解语义偏移）
- Sub-query：复合问拆成"Q1 / Q2 / Q3"分别检，后面再聚合

简单 FAQ 场景可以跳过这步。

[1] Dense 检索（语义路）

- Query 走 embedding 模型（bge-large / text-embedding-3 / bge-m3）
- 向量库 ANN 搜（FAISS / Milvus / Qdrant / pgvector），HNSW 索引
- 召回 top-50，**分数是 cosine / inner product**，范围各模型不一样
- 强项：同义词、意图、"服务器崩了"↔"服务不可用"
- 弱项：ERR_CONN_RESET_4XX、SKU-7781、条款编号——这些字面没出现就捞不到

[2] BM25 检索（字路面）

- 同一份 query **原词**丢进倒排索引（Elasticsearch / OpenSearch / Lucene 或 BM25-light）
- 召回 top-50，**分数是无界正数**，和 dense 分数量纲完全不同
- 强项：专名、ID、错误码、精确条款号
- 弱项：同义/ paraphrasing，"离职"和"辞职"字面零重叠就跪

💡 两路的失败模式是正交的——Dense 漏的 BM25 补、BM25 漏的 Dense 补，这就是 Hybrid 的本质。

[3] RRF 融合（关键一站）

为什么不用分数直接加/加权？ Dense 分在 [0.6, 0.95]，BM25 分在 [0, 15]，量纲都不一样，归一化又引入新超参——RRF 干脆**扔掉分数，只认排名**。

公式（Cormack 2009）：

$$RRF(d)=r\in R \sum k+rankr(d)1$$

- `rank_r(d)`：文档 d 在第 r 路召回里的排名（一般 1-indexed，0-indexed 就 +1）
- `k`：平滑常数，**默认 60**

k=60 是什么意思

k 值	rank1 vs rank10 差距	什么感觉
10	1.5x	头部太陡，单路第 1 几乎碾压
60	1.1x	Goldilocks，共识优先又不过压
100	1.05x	太平，头部信号被稀释

Cormack 原论文测了 $k=1\sim100$ ，60 在多个数据集上最稳；实践里 $k \in [30, 80]$ 都能打。

手写一个小例子

两路各召回 5 个，问 "cache 怎么用"：

文档	Dense 排	BM25 排	$RRF = 1/(60+d) + 1/(60+b)$
Caching patterns	1	3	$1/61 + 1/63 = 0.0322$
Cache impl	2	2	$1/62 + 1/62 = 0.0323 \leftarrow$ 两路都前 $\rightarrow$ 总分最高
Caching in .NET	5	1	$1/65 + 1/61 = 0.0318$

两路都排前的文档 RRF 总分最高——这就是"多信号共识"的直觉。

🔴 某路没召回的文档，那路贡献 0，不参与加分，不会被惩罚也不会被拉抬。

[4] Rerank 精排

RRF 出 top-100 后，再让 **CrossEncoder / bge-reranker** 把 (query, doc) 拼一起过一遍，算真相关性， $100 \rightarrow 5\sim10$ 。

为什么不直接 RRF 出 5？

- ANN 召 50 $\rightarrow$ RRF 融 100，是**粗排层**，要"准 + 覆盖"
- Rerank 是**精排层**，慢但准，只跑 top-100，成本可控
- 论文和工业界共识：**Rerank 对 RAG 最终 Faithfulness 提升比换 embedding 模型还明显**

[5] 拼 Prompt $\rightarrow$ LLM

Rerank 后的 5~10 个 chunk，按 metadata 里的 doc_title + section + page_num + text 拼进 context，Prompt 里加一句"只基于上下文回答，不知道就说不知道"，生成。

伪代码一眼版

```
代码块
1  # 1. 双路召回
2  dense_top50 = vector_db.search(embed(query), k=50)
3  bm25_top50  = bm25_index.search(query, k=50)
4
5  # 2. RRF 融合
6  k = 60
7  rrf = defaultdict(float)
8  for rank, doc in enumerate(dense_top50, start=1):
9      rrf[doc.id] += 1 / (k + rank)
10 for rank, doc in enumerate(bm25_top50, start=1):
11     rrf[doc.id] += 1 / (k + rank)
```



```
12
13 merged = sorted(rrf.items(), key=lambda x: -x[1])[:100]
14
15 # 3. Rerank 精排
16 reranked = cross_encoder.rank(query, [doc for _, doc in merged])
17 top5 = reranked[:5]
18
19 # 4. LLM
20 prompt = build_prompt(query, top5)
21 answer = llm(prompt)
```

面试追问的几个延伸点

Q: 除了 RRF 还有别的融合吗?

A: 有 **Weighted Fusion**——各自分数归一化到 [0,1] 后 $\alpha \cdot \text{dense} + (1-\alpha) \cdot \text{bm25}$ ， α 常从 0.7 起调。缺点是要归一化 + 调 α ，RRF 是 **参数-free**（除了 k），工业界更爱 RRF。

Q: k 要调吗?

A: 多数场景 60 不用动。调的时机：

- 某路 retriever 特别可信 → k 降到 30~40，让头部更陡
- 想让两路更"民主"、降低头部 dominance → k 拉到 100+
- 三路以上（dense + BM25 + SPLADE）k 也可以稍微往上调

Q: Elasticsearch / Qdrant / Milvus 都原生支持?

A: ES 8.9+ 的 `retriever` 里 `rule: rank_or_min_score + rrf`，Qdrant 的 `fusion: hybrid` 都内置 RRF，MongoDB 的 `$rankFusion` 也是 RRF——基本不用自己写，调 API 就行。

Q: Hybrid + RRF 之后还要 Rerank 吗?

A: 要。**RRF 是"多路召回的合并策略"，不是精排**。RRF 出 100 再扔 Rerank 到 5~10，是量产标准三明治：召回（宽）→ 融合（合）→ 精排（窄）→ LLM。

三、生成与评估层

怎么判断 RAG 效果好不好?

三大指标拆开看：

- **检索侧**：Recall@k（答案相关的块有没有被召回来）
- **生成侧**：Faithfulness（是否忠于检索内容，不瞎编）、Answer Relevance（是否答了问题）

- 端到端：BLEU/ROUGE 只是辅助，Ragas、ARES、TruLens 是专门框架

1. 如何减少 RAG 幻觉？

- 检索侧：提高召回覆盖率 + Rerank 过滤
- Prompt 侧：明确指令"只基于上下文回答，不知道就说不知道"
- 生成后：Self-RAG（生成时边生成边自查）、用事实一致性校验模型再过一道

上下文窗口塞不下所有召回块怎么办？

- Rerank 只留 top-N
 - 摘要压缩（用 LLM 先把块压成几句再拼）
 - 分次生成再用 LLM 合并
-

四、进阶与架构（中高级岗爱问）

Agentic RAG 和传统 RAG 区别？什么时候用？

- 传统 RAG：检索一次就完事，管线写死
- Agentic RAG：Agent 决定查几次、查哪个库（向量库 / SQL / Web / API）、要不要反思重写
- 适合：多步推理、跨多源、问题类型没法预设的场景

GraphRAG 解决什么问题？和 Vector RAG 怎么选？

- Vector RAG 找"相似"，GraphRAG 走"关系链"
- 问"X 对 Y 的影响，经过 Z"这种多跳 → GraphRAG
- 普通 FAQ / 单跳事实 → Vector RAG 就够了，Graph 建图谱成本太高

多模态 RAG 怎么做？（图片/表格/代码）

- 表格：转 Markdown 或用 Table RAG（按行列检索）
 - 图片：CLIP 类多模态 embedding，或先 OCR/ caption 再检
 - 代码：按 AST / 函数块切，embedding + BM25 双路
-

五、工程与踩坑（实战岗重点）

生产里最大的坑是什么？

- 切片切坏了（代码按 500 token 切断函数，检索回来拼不完整）

- 召回的块和问题不相关，LLM 被迫瞎编
- 增量更新：文档改一版，旧 chunk 怎么删、embedding 怎么重算
- 多租户隔离、权限过滤（检索完要再挡一层 ACL）
- 成本：rerank + LLM 两次调用，高并发下钱烧很快

有没有不用向量库的做法？

- 小数据量可以直接 BM25 + 全文索引
 - 长上下文模型兴起后，有人试"把所有文档塞上下文"——但超过几百份文档还是 RAG 稳
-

六、面试官常追的开放题

- "给你一个 10 万份文档的内部知识库，QPS 100，P95 延迟要 < 2s，你怎么设计？"（考架构权衡）
- "用户问了一个检索召回里没覆盖到的问题，RAG 应该怎么处理？"（考拒答 + fallback）
- "怎么证明你的 RAG 比直接用 GPT-4 不带检索强？"（考评估设计）